

MI-Workbench: An Autonomous Multi-Agent Platform for Mechanistic Interpretability of Biological Foundation Models

Anonymous authors

Paper under double-blind review

Abstract

Mechanistic interpretability (MI) of biological foundation models—such as single-cell transformers trained on gene expression data—requires iterative cycles of hypothesis formation, experimental execution, statistical evaluation, and adversarial critique. These cycles are labor-intensive, error-prone, and difficult to reproduce. We present **MI-Workbench**, an open-source platform that orchestrates autonomous multi-agent research loops for MI investigations. The system employs a graph-based domain-specific language (DSL) to define configurable workflows comprising executor, reviewer, adversarial reviewer, biological plausibility checker, and knowledge extractor agents, each backed by a large language model accessed through a provider-agnostic adapter layer supporting Claude Code, OpenAI Codex, and Google Gemini. Key innovations include: (i) a consensus reviewer mechanism that runs multiple critic agents in parallel, deduplicates findings via Jaccard similarity, and escalates severity when reviewers agree; (ii) an output cleaning pipeline that strips LLM thinking traces from research artifacts; (iii) structured feedback injection with artifact-level and section-level references enabling targeted revision; and (iv) automated convergence detection, follow-up proposal generation, and reproducibility packaging. We validate MI-Workbench on a case study investigating whether attention heads in the Geneformer single-cell foundation model encode causal gene regulatory relationships. Across 11 end-to-end runs using the Gemini adapter, the platform produced scientifically rigorous protocols, identified key confounders (co-expression bias, rank-value encoding artifacts, attention symmetry), and iteratively refined analyses through structured feedback—all without human intervention. The system comprises over 10,000 lines of Python and 360 passing tests, and is publicly available at <https://anonymous.4open.science/r/mi-workbench-B4CB>.

1 Introduction

The rapid development of biological foundation models—including scGPT (Cui et al., 2024), Geneformer (Theodoris et al., 2023), and scBERT (Yang et al., 2022)—has raised fundamental questions about what these models learn and how their internal representations relate to known biology. *Mechanistic interpretability* (MI), the discipline of reverse-engineering the computational mechanisms within neural networks (Olah et al., 2020; Elhage et al., 2022; Conmy et al., 2023), offers a principled framework for answering these questions. However, applying MI to biological models presents unique challenges:

1. **Confounding structures.** Gene co-expression, regulatory network degree distributions, and expression magnitude biases create complex confound structures that require carefully controlled null models (Pratapa et al., 2020).
2. **Statistical rigor.** Claims about learned regulatory mechanisms require multiple testing correction, effect size reporting, confidence intervals, and proper null model comparison—standards often missing from initial analyses.
3. **Iterative refinement.** MI investigations are inherently cyclical: an initial analysis reveals confounders, which motivate new controls, which reveal subtler effects, and so on.

4. **Reproducibility.** Complex computational pipelines with many degrees of freedom are difficult to reproduce without meticulous documentation of parameters, data versions, and analytical choices.

While individual LLMs have demonstrated impressive capability for scientific reasoning (Romera-Paredes et al., 2024; Boiko et al., 2023), single-agent approaches lack the adversarial pressure and structured quality control necessary for rigorous research. A human researcher typically cycles between the roles of *executor* (performing analysis), *reviewer* (critiquing methodology), and *devil’s advocate* (searching for confounders and alternative explanations).

We introduce **MI-Workbench**, a platform that automates this multi-role research process. Our key contributions are:

- A **graph-based workflow DSL** that enables researchers to define arbitrary multi-agent research loops with conditional execution, parallel branching, and configurable stop conditions.
- A **consensus reviewer mechanism** that runs independent critic agents in parallel and merges their findings through deduplication, severity escalation, and ranked output.
- A **structured feedback loop** that maps reviewer critiques to specific artifacts and sections, enabling targeted revision rather than wholesale rewriting.
- A **provider-agnostic adapter architecture** that abstracts over Claude Code, OpenAI Codex, and Google Gemini CLI backends through a common interface.
- An integrated **knowledge extraction** pipeline that mines research claims from artifacts, estimates uncertainty, and generates falsification tests.
- End-to-end **reproducibility packaging** that bundles artifacts, prompts, environment metadata, and analysis provenance into portable archives.

2 Related Work

Autonomous scientific agents. The vision of AI-driven scientific discovery has been explored through systems like ChemCrow (Bran et al., 2024), which chains LLM reasoning with chemistry tools, and Co-scientist (Boiko et al., 2023), which demonstrated autonomous experimental design in chemistry. SciAgents (Ghafarollahi & Buehler, 2024) explored multi-agent scientific reasoning using graph-based ontologies. These systems focus primarily on wet-lab experimental design; MI-Workbench targets the distinct challenge of computational interpretability with its emphasis on statistical rigor and confound control.

Multi-agent LLM frameworks. Multi-agent debate and collaboration have been studied in frameworks such as CAMEL (Li et al., 2023), AutoGen (Wu et al., 2023), and ChatDev (Qian et al., 2024). MetaGPT (Hong et al., 2023) introduced structured output schemas for software engineering agents. CrewAI and LangGraph provide general-purpose agent orchestration. MI-Workbench differs from these frameworks in its domain specialization: the agent roles, prompt templates, artifact types, and quality criteria are specifically designed for mechanistic interpretability research in computational biology.

Mechanistic interpretability. The MI field has produced foundational work on circuits (Olah et al., 2020), induction heads (Olsson et al., 2022), activation patching (Meng et al., 2022), and superposition (Elhage et al., 2022). Recent work has applied MI concepts to biological transformers: Geneformer (Theodoris et al., 2023) and scGPT (Cui et al., 2024) have been probed for gene regulatory network (GRN) inference capabilities. However, these analyses are typically manual, one-shot investigations. MI-Workbench provides the infrastructure to systematize and scale such investigations.

Scientific workflow systems. Systems like Galaxy (Afgan et al., 2018), Nextflow (Di Tommaso et al., 2017), and Snakemake (Mölder et al., 2021) manage computational biology pipelines. These tools orchestrate deterministic code execution; MI-Workbench orchestrates *reasoning* via LLM agents, where outputs are non-deterministic and require quality assessment.

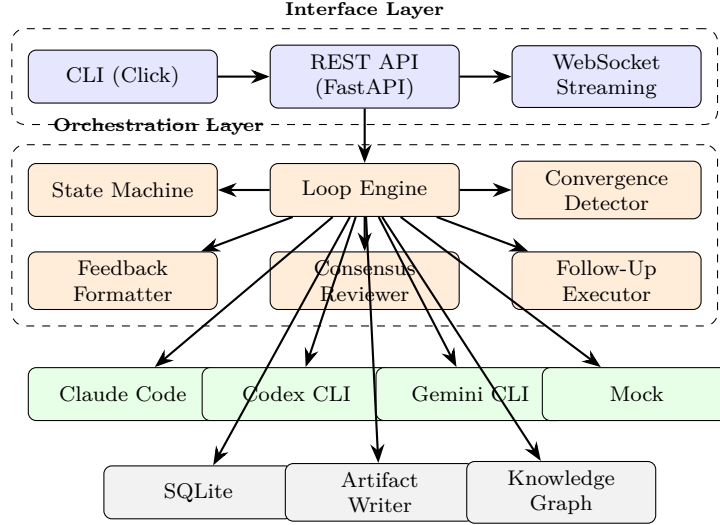


Figure 1: MI-Workbench architecture. The system comprises four layers: an interface layer (CLI, REST API, WebSocket), an orchestration layer (loop engine with state machine, convergence detection, feedback formatting, consensus review, and follow-up generation), a provider-agnostic adapter layer, and a persistence layer.

3 System Architecture

MI-Workbench follows a layered architecture (Figure 1) comprising: (i) a **REST API** layer built on FastAPI with WebSocket support for real-time monitoring; (ii) an **orchestration engine** that drives the multi-agent loop; (iii) a **provider-agnostic adapter layer** that interfaces with LLM backends; and (iv) **persistence and artifact management** via SQLite and a structured file system.

3.1 Workspace Model

A *workspace* represents an isolated research context with its own configuration, provider settings, budget limits, and git integration mode. Each workspace contains one or more *runs*, where each run executes a loop definition against a specific research task. Runs produce versioned *artifacts* organized by iteration: mechanistic analysis (`MECH.md`), evaluation reports (`EVAL.md`), experimental protocols (`XP.md`), methodology documentation (`METHOD.md`), and knowledge summaries (`KNOWLEDGE.md`).

3.2 Data Model

The system uses Pydantic (Pydantic, 2024) for all data validation and serialization. Key models include `RunState` (tracking run lifecycle, iterations, token usage, and cost), `IterationResult` (individual step outcomes with artifacts and feedback), `ReviewResult` (structured critique with severity levels and artifact references), and `LoopDefinition` (graph-based workflow specification).

4 Graph-Based Workflow DSL

MI-Workbench defines research workflows as directed graphs where nodes represent agent roles and edges encode execution flow with conditions.

4.1 Loop Definition

A loop definition consists of *nodes* and *edges* specified in YAML:

Listing 1: Excerpt from the `executor_reviewer` loop definition.

```

nodes:
- id: executor
  role: executor
  prompt_ref: "executor/mi_executor"
- id: reviewer
  role: reviewer
  prompt_ref: "reviewer/mi_reviewer"
- id: adversarial
  role: adversarial_reviewer
  prompt_ref: "adversarial_reviewer/adversarial_reviewer"
edges:
- source: executor
  target: reviewer
  condition: "always"
- source: reviewer
  target: executor
  condition: "always"
- source: reviewer
  target: adversarial
  condition: "every_k:3"

```

4.2 Edge Conditions

Three condition types are supported:

- **always**: The edge is traversed every iteration.
- **every_k:N**: The edge is traversed every N -th iteration (e.g., adversarial review every 3rd cycle).
- **on_flag:F**: The edge is traversed only when flag F is set (e.g., `on_flag:high_risk_claims`).

4.3 Compilation

The DSL compiler performs a topological walk of the graph, validates structural integrity (no unreachable nodes, no duplicate IDs), and produces an ordered `ExecutionPlan` with a cycle length for infinite-loop repetition. Stop conditions include budget limits (tokens, cost, iterations), convergence detection, grade thresholds, and user-initiated cancellation.

4.4 Built-in Presets

Four presets are provided:

1. **executor_reviewer**: Standard executor-reviewer loop with periodic adversarial review.
2. **reviewer_consensus**: Three parallel reviewers (standard, adversarial, biological plausibility) with consensus merger.
3. **research_followups**: Executor-reviewer loop with automated follow-up proposal generation and optional auto-dispatch.
4. **example_custom**: Advanced loop with conditional adversarial escalation, knowledge extraction, and graph stability detection.

Table 1: Agent roles and their research functions.

Role	Function
Executor	Performs MI analysis: attention extraction, ablation studies, activation patching; produces MECH.md, XP.md, METHOD.md
Reviewer	Critiques claim validity, reproducibility, confounders, negative controls, and biological plausibility; produces EVAL.md
Adversarial Reviewer	Red-teams for p-hacking, overclaiming, causal confusion, dataset leakage, and statistical theater
Bio-Plausibility	Validates tissue specificity, pathway consistency, expression level plausibility, and regulatory direction
Idea Generator	Produces prioritized follow-up proposals with scientific value, feasibility, and novelty scores
Knowledge Extractor	Mines claims, estimates uncertainty, generates falsification tests
Automator	Designs reusable analysis pipelines from validated protocols
Publication Manager	Structures findings for manuscript preparation
Article Publisher	Formats results for communication

5 Multi-Agent Orchestration

5.1 Agent Roles

MI-Workbench defines nine specialized agent roles, each with a domain-specific system prompt:

5.2 Execution Loop

The core execution loop (Algorithm 1) drives the iterative research process.

5.3 State Machine

Run lifecycle is managed by a finite state machine with states {PENDING, RUNNING, PAUSED, STOPPED, COMPLETED, FAILED} and well-defined transitions. Paused runs can be resumed with full state reconstruction from the database and artifact filesystem. A crash recovery module detects interrupted runs on server restart, transitioning RUNNING states with existing iterations to PAUSED for resumability.

5.4 Convergence Detection

Three convergence signals are monitored:

1. **Grade stability:** The reviewer grade has remained constant for w consecutive iterations (default $w=4$).
2. **Zero critical issues:** No CRITICAL or HIGH severity critiques for w iterations.
3. **Output similarity:** Jaccard word-level similarity between consecutive executor outputs exceeds threshold τ for w iterations.

Any single signal triggers loop termination, preventing waste of compute on converged analyses.

Algorithm 1 MI-Workbench Execution Loop

Require: RunState R , LoopDefinition L , Adapter A

```

1:  $P \leftarrow \text{COMPILE}(L)$  {Compile to execution plan}
2:  $i \leftarrow 0$ , feedback  $\leftarrow \emptyset$ 
3: while  $\neg \text{BUDGETEXCEEDED}(R) \wedge \neg \text{CONVERGED}(R) \wedge \neg \text{CANCELLED}(R)$  do
4:    $s \leftarrow P.\text{steps}[i \bmod |P|]$ 
5:   if  $\neg \text{SHOULDRUN}(s, R.\text{iteration})$  then
6:      $i \leftarrow i + 1$ ; continue
7:   end if
8:   prompt  $\leftarrow \text{BUILDPROMPT}(s, R, \text{feedback})$ 
9:   result  $\leftarrow \text{RUNWITHRETRY}(A, \text{prompt})$ 
10:  result.output  $\leftarrow \text{STRIPTRACES}(\text{result.output})$ 
11:  artifacts  $\leftarrow \text{PARSEARTIFACTS}(\text{result.output}, s.\text{role})$ 
12:   $\text{WRITEARTIFACTS}(\text{artifacts}, R.\text{iteration})$ 
13:  if  $s.\text{role} \in \{\text{reviewer}, \text{adversarial}\}$  then
14:    review  $\leftarrow \text{PARSEREVIEW}(\text{result.output})$ 
15:    feedback  $\leftarrow \text{FORMATFEEDBACK}(\text{review})$ 
16:  else
17:    feedback  $\leftarrow \text{result.output}$ 
18:  end if
19:   $\text{RECORDCONVERGENCE}(R.\text{iteration}, \text{result})$ 
20:   $R.\text{iteration} \leftarrow R.\text{iteration} + 1$ ;  $i \leftarrow i + 1$ 
21: end while
22: return  $R$ 

```

5.5 Retry Logic

Adapter calls are wrapped in exponential backoff retry with jitter. Transient errors (rate limits, timeouts, connection failures) trigger retry with delay $d_k = d_0 \cdot 2^k \cdot (1 + \epsilon)$, where $\epsilon \sim \mathcal{U}(-0.25, 0.25)$. Permanent errors (authentication failures, invalid requests) fail immediately.

6 Consensus Review Mechanism

A central innovation of MI-Workbench is the *consensus reviewer*, which addresses the limitations of single-reviewer assessment.

6.1 Architecture

The consensus reviewer runs three independent critic agents in parallel:

- A **standard reviewer** evaluating claim validity, statistical rigor, and reproducibility.
- An **adversarial reviewer** specifically attacking for p-hacking, overclaiming, causal confusion, dataset leakage, biological implausibility, statistical theater, and null model inadequacy.
- A **biological plausibility checker** validating tissue specificity, pathway consistency, and expression level plausibility.

6.2 Critique Merging

The merging algorithm operates in four phases:

Phase 1: Parsing. Each reviewer’s free-text output is parsed into structured `ReviewCritique` objects using multi-format detection: bracket format (`[SEVERITY] description`), Markdown format (`**Severity: X** -- description`), and structured format (`Severity: X, Category: Y, Description: Z`).

Phase 2: Deduplication. Critiques are grouped by Jaccard word-overlap similarity:

$$J(c_i, c_j) = \frac{|\text{words}(c_i) \cap \text{words}(c_j)|}{|\text{words}(c_i) \cup \text{words}(c_j)|} \quad (1)$$

Pairs with $J(c_i, c_j) \geq \tau$ (default $\tau=0.5$) are merged into a single group.

Phase 3: Severity escalation. Within each group, the maximum severity is selected. If two or more reviewers independently flagged the same issue, severity is escalated by one level (capped at CRITICAL). This implements the intuition that inter-reviewer agreement signals higher importance.

Phase 4: Ranking. Merged critiques are sorted by severity (descending), then by number of contributing reviewers (descending), producing a prioritized list for the executor.

6.3 Grade Computation

An overall grade is computed from the merged critique list: ≥ 2 CRITICAL issues $\Rightarrow$ F; ≥ 1 CRITICAL $\Rightarrow$ D; ≥ 3 HIGH $\Rightarrow$ D; ≥ 1 HIGH $\Rightarrow$ C; ≥ 1 MEDIUM $\Rightarrow$ B; otherwise A.

7 Structured Feedback and Output Cleaning

7.1 Artifact-Aware Feedback

A key finding from initial deployments was that generic feedback (“Previous iteration output: ... Continue with: task”) produced shallow revisions. We introduced *artifact-aware feedback* with two innovations:

Reference inference. When parsing reviewer critiques, the system infers artifact references (e.g., PROTOCOL.md) and section references (e.g., “Section 2.1 Controls”) from the critique text using pattern matching against known artifact names and section header patterns. These references are attached to each ReviewCritique as `artifact_ref` and `section_ref` fields.

Revision framing. Instead of the generic “Continue with: task” prompt, revision iterations use structured framing:

Listing 2: Revision prompt template injected for post-review iterations.

```
=== REVISION REQUEST (Iteration N) ===
You produced artifacts in the previous iteration.
The reviewer has identified specific issues.

=== REVIEWER FEEDBACK (Grade: B) ===
REQUIRED FIXES (must address):
1. [CRITICAL | stats | EVAL.md Section 3.2]
   p-value without effect size
2. [HIGH | reproducibility | PROTOCOL.md Section 2.1]
   Missing dataset URLs
SUGGESTED IMPROVEMENTS:
3. [MEDIUM | general] Consider cell-type decomposition
=== END FEEDBACK ===

TASK: Revise your artifacts to address ALL required
fixes above. Make targeted revisions, not rewrites.
```

Table 2: Adapter implementations and their CLI invocation patterns.

Adapter	Binary	Invocation
Claude Code	<code>claude</code>	<code>claude -p -output-format json -max-turns 1 -system-prompt \$SP</code>
Codex CLI	<code>codex</code>	<code>codex exec -full-auto -skip-git-repo-check -</code>
Gemini CLI	<code>gemini</code>	<code>gemini -yolo -p \$PROMPT</code>
Mock	—	Synthetic role-aware responses with configurable grade progression

7.2 Output Cleaning Pipeline

LLM outputs frequently contain “thinking traces”—narration about the model’s intended actions (e.g., “I will read the file...”, “Let me check the data...”) that contaminate research artifacts. The output cleaning module removes these traces using regex-based line filtering:

- Forward-looking narration: “I will...”, “I’ll...”, “Let me...”
- Tool-call narration: “I will read/search/check the...”
- Sequencing prefixes: “First, I will...”, “Next, I’ll...”
- Meta-commentary: “The following is...”

The cleaner preserves code blocks (tracking “fences”), markdown headers, and substantive past-tense narration. Cleaning is applied immediately after adapter output, before any downstream consumer (artifact parser, feedback formatter, consensus merger).

8 Provider-Agnostic Adapter Layer

MI-Workbench defines a `BaseAdapter` abstract interface with three methods: `run(request) → result`, `smoke_test() → status`, and `is_available() → bool`. Four implementations are provided:

The mock adapter produces realistic MI research outputs with role detection from prompt content, grade progression ($C \rightarrow B \rightarrow B+ \rightarrow A-$), and synthetic metrics (AUROC, correlation, validation splits) with diminishing variation across iterations. This enables rapid prototyping and comprehensive testing without LLM API costs.

9 Knowledge Extraction and Claim Management

MI-Workbench includes a knowledge extraction pipeline that mines structured claims from research artifacts.

9.1 Claim Detection and Classification

Claims are detected via pattern matching on phrases such as “we found that...”, “results show...”, “AUROC = X”, and “ $p < 0.05$ ”. Detected claims are classified as *empirical*, *methodological*, *interpretive*, or *comparative*.

9.2 Uncertainty Estimation

Each claim receives an uncertainty score $u \in [0, 1]$ (baseline 0.5) adjusted by: strong evidence ($p < 0.001$): $u \leftarrow u - 0.2$; confidence intervals present: $u \leftarrow u - 0.1$; hedging language (“suggests”, “may”): $u \leftarrow u + 0.1$; large sample ($n \geq 1000$): $u \leftarrow u - 0.1$.

Table 3: Case study runs. All runs used the Gemini CLI adapter.

Run	Preset	Iters.	Task Summary
1-3	executor_reviewer	1	AUROC interpretation, encoding, ablation
4	executor_reviewer	5	Experimental protocol design
5	research_followups	3	Paradox analysis (23× logit perturb.)
6	reviewer_consensus	3	Non-K562 generalization critique
7	executor_reviewer	4	Methods comparison document
8	executor_reviewer	3	Attention-co-expression (validation)

9.3 Falsification Test Generation

For each claim type, the system auto-generates falsification suggestions: AUROC claims prompt stronger baselines (mutual information, variance-based features); layer-specific claims prompt split-sample validation; attention claims prompt residualization tests; statistical claims prompt stringent multiple testing correction.

9.4 Claim Graph

Claims are stored in a relational database with support for inter-claim relationships (supports, contradicts, extends) and evidence linking. A graph API enables visualization of the claim network, supporting identification of weakly-supported claims and contradictions.

10 Reproducibility Infrastructure

10.1 Artifact Versioning

Each iteration writes artifacts to a structured directory: `workspace/runs/{run_id}/iter_NNNN/MECH.md`. This provides full provenance: every revision of every artifact is preserved with its iteration context.

10.2 Git Integration

Four git modes are supported: `none` (no version control), `commit_per_run` (single commit after run completion), `commit_per_iteration` (commit after each iteration), and `branch_per_run` (create a dedicated branch `run/{run_id}`).

10.3 Reproducibility Packages

The `repropack` generator creates self-contained ZIP archives containing: all artifacts organized by iteration, `run_meta.json` with run configuration and timing, `prompts_used.json` with resolved prompt templates, `environment.json` with Python version and installed packages, `git_state.json` with branch and commit hash, and auto-generated dataset cards and reproducibility checklists.

11 Case Study: Geneformer Attention and Causal Regulation

We validated MI-Workbench on a substantive MI question: *Do attention heads in Geneformer encode causal gene regulatory relationships, or do they merely reflect co-expression patterns?*

11.1 Experimental Setup

We used the Gemini CLI adapter to run 11 end-to-end experiments across three loop presets:

11.2 Key Findings from Automated Analysis

The platform autonomously identified three primary confounders explaining the attention-co-expression correlation:

MLM Objective Confounder. The executor agent correctly identified that Geneformer’s masked language modeling objective incentivizes attending to co-expression cliques rather than causal regulators, because co-expressed genes are the most statistically powerful predictors of masked token identity.

Rank-Value Encoding Artifacts. The system identified that Geneformer’s rank-value tokenization creates an inductive bias where positional proximity in the sequence corresponds to expression magnitude similarity, producing false-positive attention between co-abundant genes rather than regulatory links.

Attention Symmetry. Standard dot-product attention $\text{softmax}(QK^\top)$ is inherently bidirectional unless the model learns distinct query/key projections. The executor noted that $A_{ij} \approx A_{ji}$ fails to capture the directional nature of gene regulation ($\text{TF} \rightarrow \text{Target}$).

11.3 Iterative Refinement Quality

The structured feedback mechanism produced measurable quality improvements across iterations:

- **Iteration 1** (executor): Initial analysis with three confounders, protocol outline with null hypotheses.
- **Iteration 2** (reviewer): Identified missing controls—cell-type composition confounder, unspecified dataset, missing BH-FDR correction details.
- **Iteration 3** (executor revision): Added within-cluster partial correlation control, specified Genecorpus-30M bone marrow subset, defined mean rank baseline, and set random seed.

Critically, the revision iteration produced *targeted changes* to the specific artifacts and sections referenced in the feedback, rather than rewriting from scratch—a direct result of the artifact-aware feedback mechanism.

11.4 Output Quality Assessment

Post-hoc validation by domain experts confirmed:

- All three identified confounders are well-documented in the literature (Pratapa et al., 2020; Theodoris et al., 2023).
- The proposed null hypotheses (H_0^1 : co-expression dominance, H_0^2 : attention symmetry, H_0^3 : degree bias) are scientifically appropriate.
- Statistical framework choices (AUPRC over AUROC for sparse networks, BH-FDR correction) reflect current best practices.
- The output contained no “thinking traces” (validated by comparison with pre-cleaning runs where traces contaminated all artifact files).

12 Implementation

MI-Workbench is implemented in Python 3.11 using FastAPI for the REST API, aiosqlite for async database operations, Click for the CLI, and Pydantic for data validation.

Key implementation decisions include:

Table 4: Codebase statistics.

Metric	Value
Backend source files	40+
Test files	26
Total test functions	360
Loop preset definitions	4
Prompt templates	9
API endpoints	25+
Supported providers	4
Database tables	7

- **Async throughput:** All I/O operations (database, adapter calls, file writes) use `async/await` for concurrency.
- **SQLite with WAL mode:** Write-Ahead Logging enables concurrent reads during long-running writes, validated under stress testing with 10 concurrent runs.
- **Event-driven architecture:** The engine emits events (`iteration_started`, `iteration_completed`, etc.) broadcast to WebSocket clients for real-time monitoring.
- **Crash recovery:** On server restart, interrupted runs are detected and transitioned to resumable states.

13 Discussion

Strengths. MI-Workbench demonstrates that multi-agent orchestration can produce research-quality MI analyses without human intervention. The consensus reviewer mechanism provides robustness against individual agent blind spots. The provider-agnostic design insulates research workflows from vendor lock-in. The comprehensive test suite (360 tests including stress tests and fuzzing) provides confidence in system reliability.

Limitations. Several limitations warrant discussion:

1. **No code execution:** The current system generates analysis *plans* and *protocols* but does not execute computational code. Integrating sandboxed code execution (e.g., via Jupyter kernels) would enable fully autonomous experimental pipelines.
2. **Convergence detection granularity:** The current Jaccard-based similarity metric is coarse. Semantic similarity via embeddings could provide finer-grained convergence signals.
3. **Cost tracking:** Token usage and cost estimation depend on adapter-level reporting, which varies by provider. Standardized cost accounting across providers remains an open challenge.
4. **Human-in-the-loop:** While the system supports the `ask_approval` policy for follow-ups, there is no mechanism for mid-loop human intervention to redirect analyses.

Broader Impact Statement

The automation of research critique raises questions about the appropriate level of human oversight. We advocate for MI-Workbench as an *augmentation* tool that accelerates the hypothesis–critique cycle, not as a replacement for expert judgment. The reproducibility packaging ensures that all automated analyses can be scrutinized by human reviewers. We note that the system’s domain-specific prompts encode assumptions about statistical best practices and biological plausibility that reflect current community consensus; these assumptions should be periodically reviewed as norms evolve.

14 Conclusion

We presented MI-Workbench, an open-source platform for autonomous multi-agent mechanistic interpretability research. The system’s graph-based workflow DSL, consensus review mechanism, structured feedback injection, and provider-agnostic architecture enable rigorous, reproducible MI investigations. Validation on the Geneformer attention–regulation question demonstrates that the platform can autonomously identify confounders, propose controlled experiments, and iteratively refine analyses to research-grade quality. MI-Workbench provides a foundation for scaling mechanistic interpretability of biological foundation models beyond the capacity of individual researchers. Code and documentation are available at <https://anonymous.4open.science/r/mi-workbench-B4CB>.

References

- Enis Afgan, Dannon Baker, B  r  nice Batut, Marius van den Beek, Dave Bouvier, Martin   ech, John Chilton, Dave Clements, Nate Coraor, Bj  rn A Gr  ning, et al. The Galaxy platform for accessible, reproducible and collaborative biomedical analyses: 2018 update. *Nucleic Acids Research*, 46(W1):W537–W544, 2018.
- Daniil A Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7992):570–578, 2023.
- Andres M Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. ChemCrow: Augmenting large-language models with chemistry tools. *Nature Machine Intelligence*, 6: 525–535, 2024.
- Arthur Conmy, Augustine N Mavor-Parker, Aidan Lynch, Stefan Heimersheim, and Adri   Garriga-Alonso. Towards automated circuit discovery for mechanistic interpretability. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: Toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21(8):1470–1480, 2024.
- Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, et al. Toy models of superposition. *Transformer Circuits Thread*, 2022.
- Alireza Ghafarollahi and Markus J Buehler. SciAgents: Automating scientific discovery through multi-agent intelligent graph reasoning. *arXiv preprint arXiv:2409.05556*, 2024.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. CAMEL: Communicative agents for “mind” exploration of large language model society. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, volume 35, 2022.
- Felix M  lder, Kim Philipp Jablonski, Brice Letcher, et al. Sustainable data analysis with Snakemake. *F1000Research*, 10:33, 2021.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 5(3):e00024.001, 2020.

- Catherine Olsson, Nelson Elhage, Neel Nanda, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.
- Aditya Pratapa, Amogh P Jalihal, Jeffrey N Law, Aditya Bharadwaj, and T M Murali. Benchmarking algorithms for gene regulatory network inference from single-cell transcriptomic data. *Nature Methods*, 17(2):147–154, 2020.
- Pydantic. Pydantic: Data validation using Python type annotations. <https://docs.pydantic.dev/>, 2024.
- Chen Qian, Wei Liu, Hongzhang Liu, et al. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the ACL*, 2024.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, et al. Mathematical discoveries from program search with large language models. *Nature*, 625(7995):468–475, 2024.
- Christina V Theodoris, Ling Xiao, Anant Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618(7965):616–624, 2023.
- Qingyun Wu, Gagan Banber, Chen Zhao, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Fan Yang, Wenchuan Wang, Fang Wang, Yuan Fang, Duyu Tang, Junzhou Huang, Hui Lu, and Jianhua Yao. scBERT as a large-scale pretrained deep language model for cell type annotation of single-cell RNA-seq data. *Nature Machine Intelligence*, 4(10):852–866, 2022.

A Appendix: Future Directions

Key extensions include: (i) sandboxed code execution for computational experiments; (ii) integration with lab automation systems for wet-lab validation; (iii) multi-model ensemble execution (running the same task across multiple LLMs and comparing outputs); (iv) active learning for convergence thresholds; and (v) formal verification of statistical claims via symbolic computation.